

Tech & Teach

Digital Chip Design Program

RTL Design

Almas Alshubrumi

Atheer Al-harbi

Mjd AL-harbi

Maha Omar

Jood Alomair

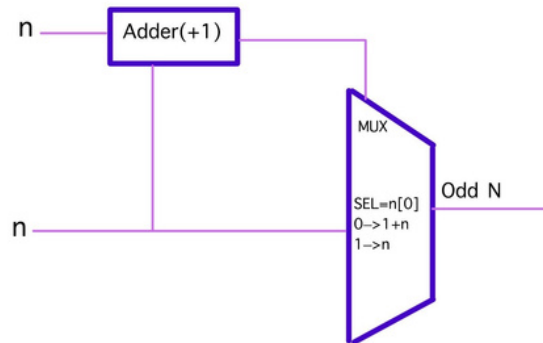
8/7/2026

Combinational Datapath Design Part 2

Tech & Teach

Task #6.1

1.1



```
module toOdd #(parameter WIDTH = 4)
```

```
(
```

```
  input logic [WIDTH-1:0] n,
```

```
  output logic [WIDTH-1:0] oddN
```

```
);
```

```
logic [WIDTH-1:0] w;
```

```
adder_subtractor #(WIDTH) addsub(
```

```
  .A(n),
```

```
  .B(1),
```

```
  .M(0),
```

```
  .S(w)
```

```
);
```

```
mux2to1 #(WIDTH) mux0(
```

```
  .S(~n[0]),
```

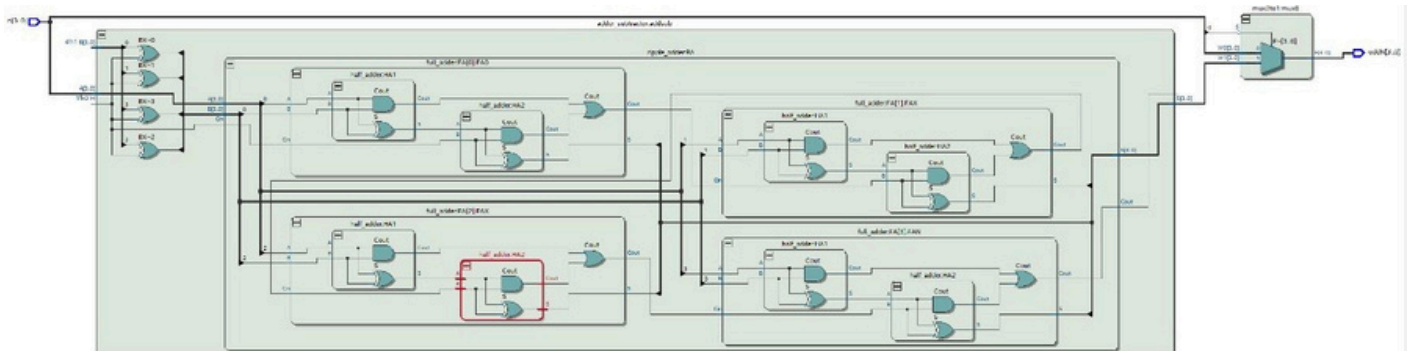
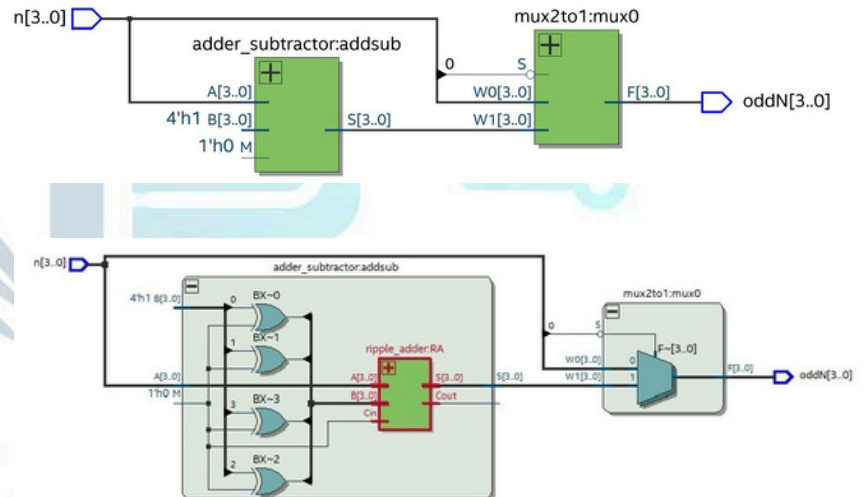
```
  .W0(n),
```

```
  .W1(w),
```

```
  .F(oddN)
```

```
);
```

```
endmodule
```



Task #6.1

1.1

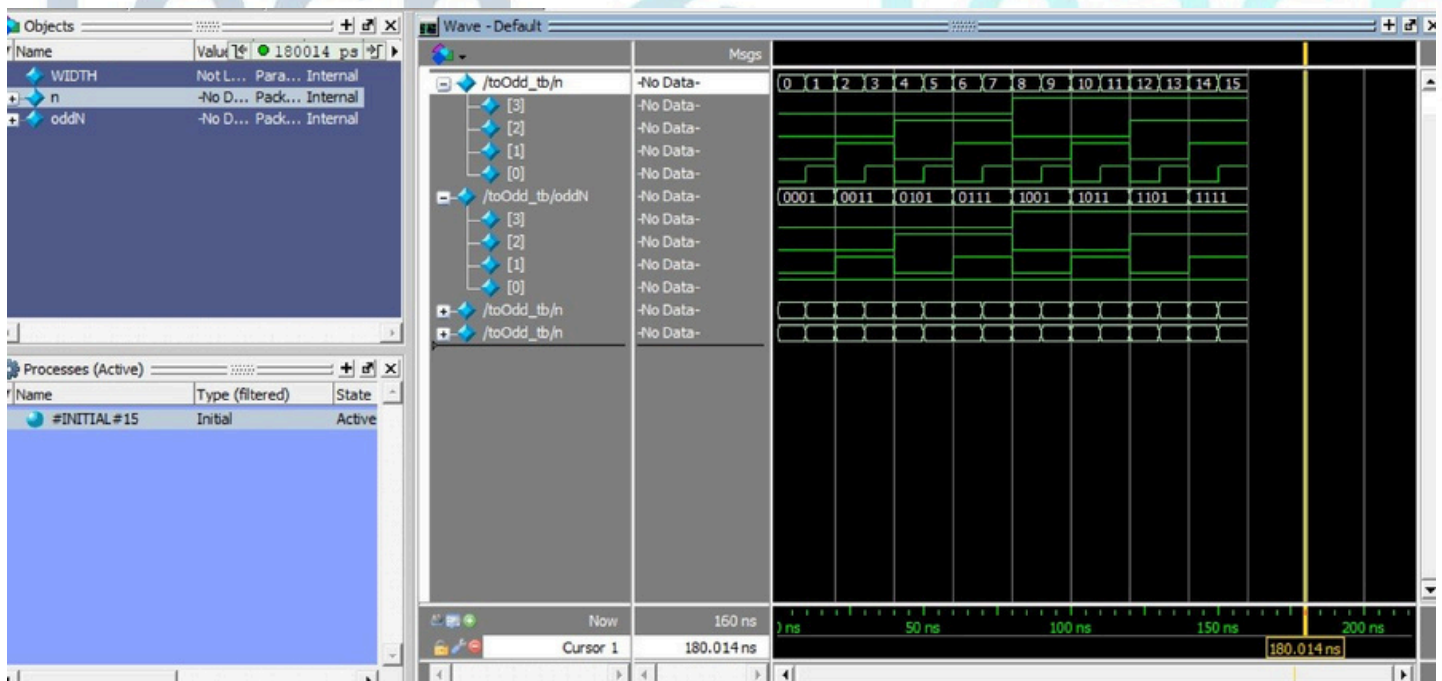
```
`timescale 1ns/1ps
module toOdd_tb;
parameter WIDTH = 4;
logic [WIDTH-1:0] n;
logic [WIDTH-1:0] oddN;
toOdd #(WIDTH) DUT(
    .n(n),
    .oddN(oddN)
);
```

```
initial begin
    n = 4'd0; #10;
    n = 4'd1; #10;
    n = 4'd2; #10;
    n = 4'd3; #10;
    n = 4'd4; #10;
    n = 4'd5; #10;
    n = 4'd6; #10;
    n = 4'd7; #10;
    n = 4'd8; #10;
    n = 4'd9; #10;
    n = 4'd10; #10;
    n = 4'd11; #10;
    n = 4'd12; #10;
    n = 4'd13; #10;
    n = 4'd14; #10;
    n = 4'd15; #10;
```

```
$finish;
```

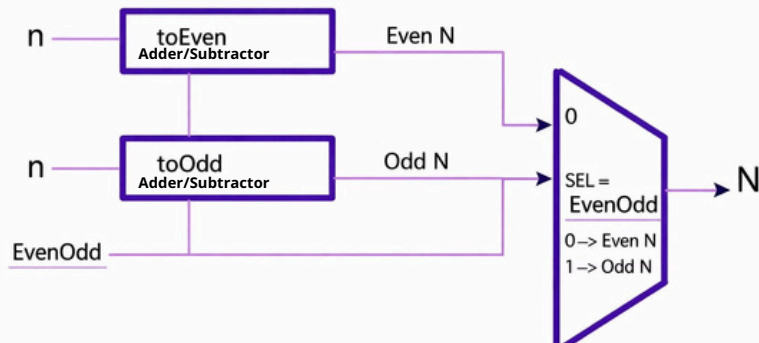
```
end
```

```
endmodule
```



Task #6.1

1.2



```
module toEvenOdd #(parameter WIDTH = 4)
```

```
(
  input logic [WIDTH-1:0] n,
  input logic EvenOdd,
  output logic [WIDTH-1:0] N
```

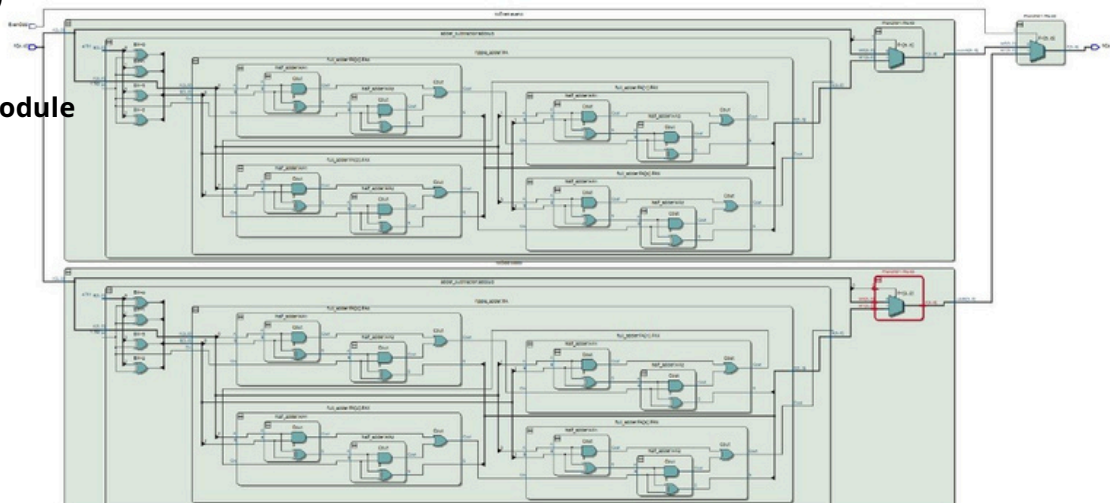
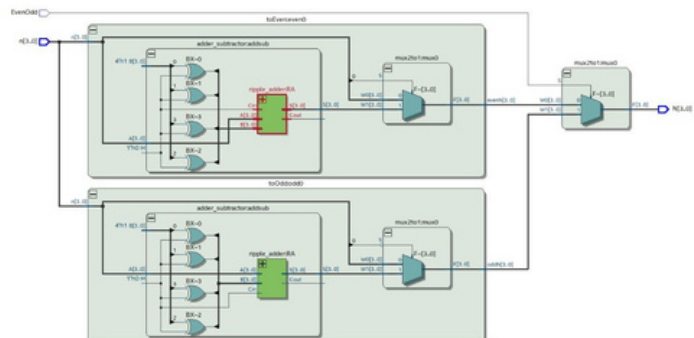
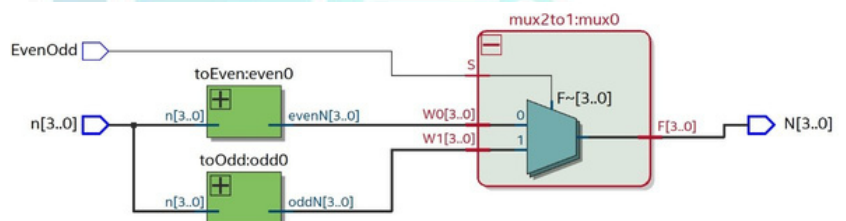
```
);
logic [WIDTH-1:0] evenOut;
logic [WIDTH-1:0] oddOut;
```

```
toEven #(WIDTH) even0(
  .n(n),
  .evenN(evenOut)
```

```
);
toOdd #(WIDTH) odd0(
  .n(n),
  .oddN(oddOut)
```

```
);
mux2to1 #(WIDTH) mux0(
  .S(EvenOdd),
  .W0(evenOut),
  .W1(oddOut),
  .F(N)
```

```
);
endmodule
```



Task #6.1

1.2

```
`timescale 1ns/1ps
```

```
module toEvenOdd_tb;
```

```
parameter WIDTH = 4;
```

```
logic [WIDTH-1:0] n;
```

```
logic EvenOdd;
```

```
logic [WIDTH-1:0] N;
```

```
toEvenOdd #(WIDTH) D(
```

```
  .n(n),
```

```
  .EvenOdd(EvenOdd),
```

```
  .N(N)
```

```
);
```

```
initial begin
```

```
  // Even
```

```
  EvenOdd = 0;
```

```
  n = 4'b0001; #20;
```

```
  n = 4'b0010; #20;
```

```
  n = 4'b0011; #20;
```

```
  n = 4'b0100; #20;
```

```
  // Odd
```

```
  EvenOdd = 1;
```

```
  n = 4'b0001; #20;
```

```
  n = 4'b0010; #20;
```

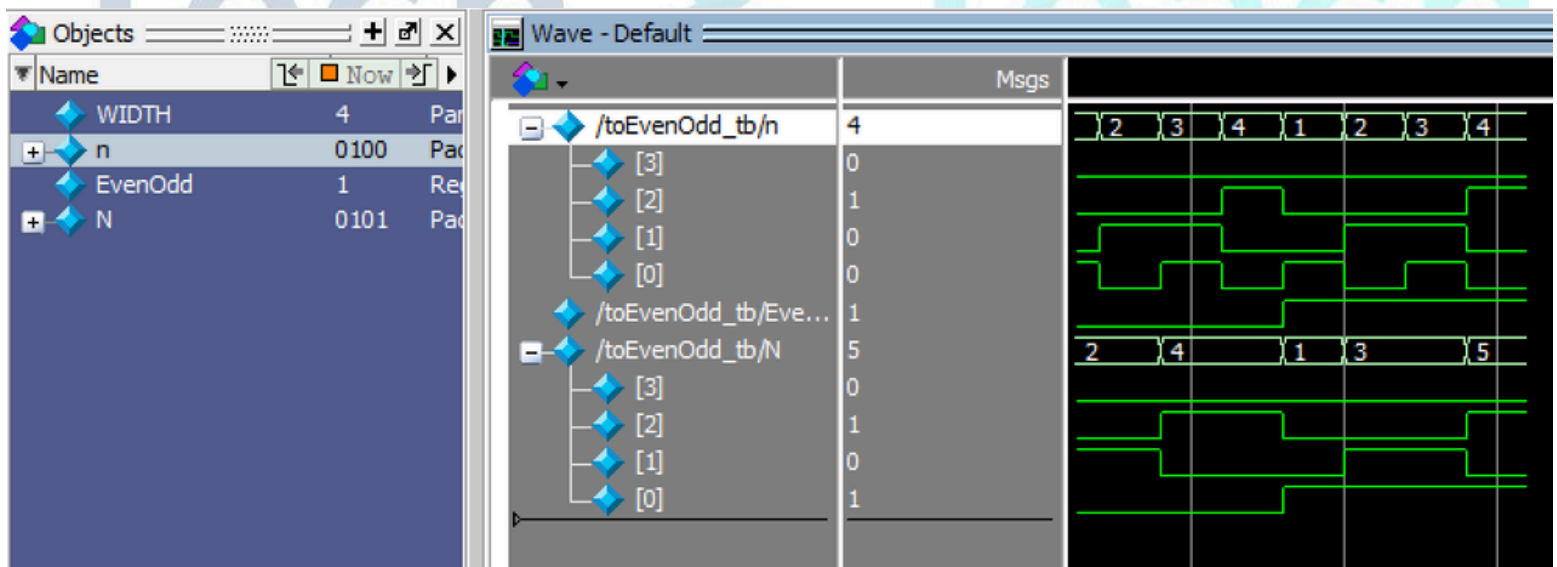
```
  n = 4'b0011; #20;
```

```
  n = 4'b0100; #20;
```

```
  $finish;
```

```
end
```

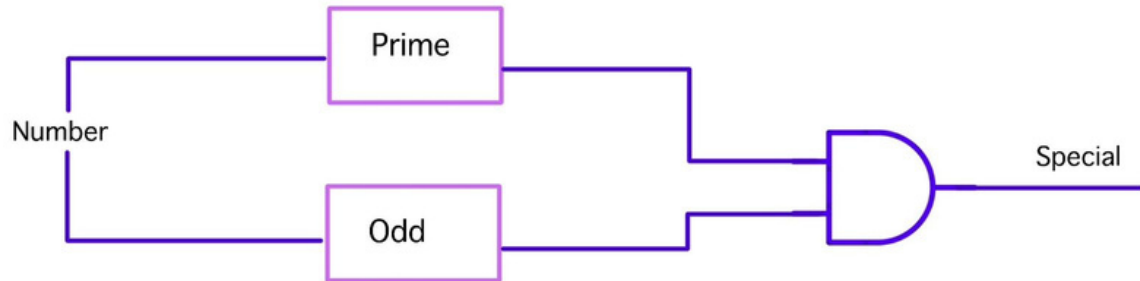
```
endmodule
```



Task #6.2

2.1 input logic [N-1:0] number;
output logic special;

2.2



2.3

```
module isspecial #(
    parameter int N = 8
) (
    input logic [N-1:0] number,
    output logic      special
);

    logic is_odd;
    logic is_prime;
    int i;

    assign is_odd = number[0];

    always_comb begin
        is_prime = 1'b1;

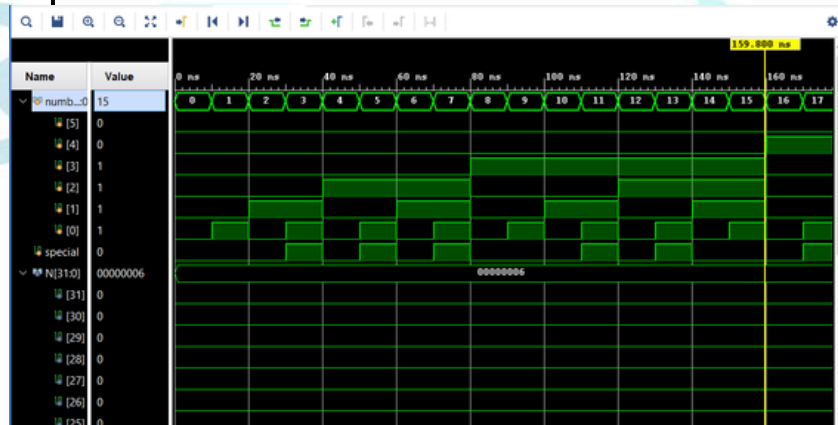
        if (number < 2) begin
            is_prime = 1'b0;
        end
        else begin
            for (i = 2; i < (1 << N); i++) begin
                if ((i < number) && (number % i == 0)) begin
                    is_prime = 1'b0;
                end
            end
        end
    end

    assign special = is_odd & is_prime;
endmodule
```

2.4

```
`timescale 1ns / 1ps

module isspecial_tb();
    parameter N=6 ;
    logic [N-1:0] number ;
    logic special ;
    isspecial #(.N(N)) dut (
        .number(number),
        .special(special));
    initial begin
        for (int val = 0; val < (2**N); val++) begin
            number = val[N-1:0];
            #10;
        end
        $finish;
    end
endmodule
```

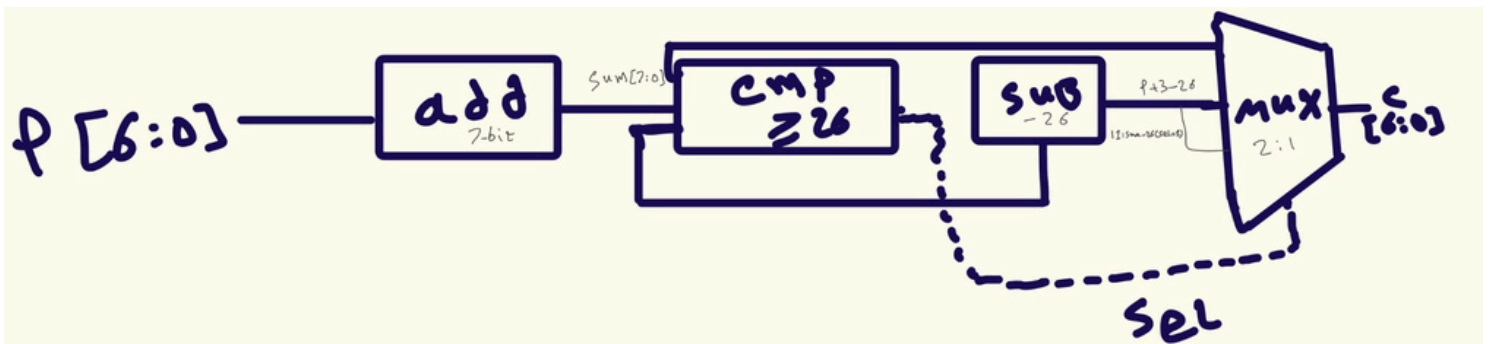


Task #6.2

3.1

- Input: $p[6:0]$ (7-bit ASCII)
- Output: $c[6:0]$ (7-bit ASCII بعد التشفير)

3.2

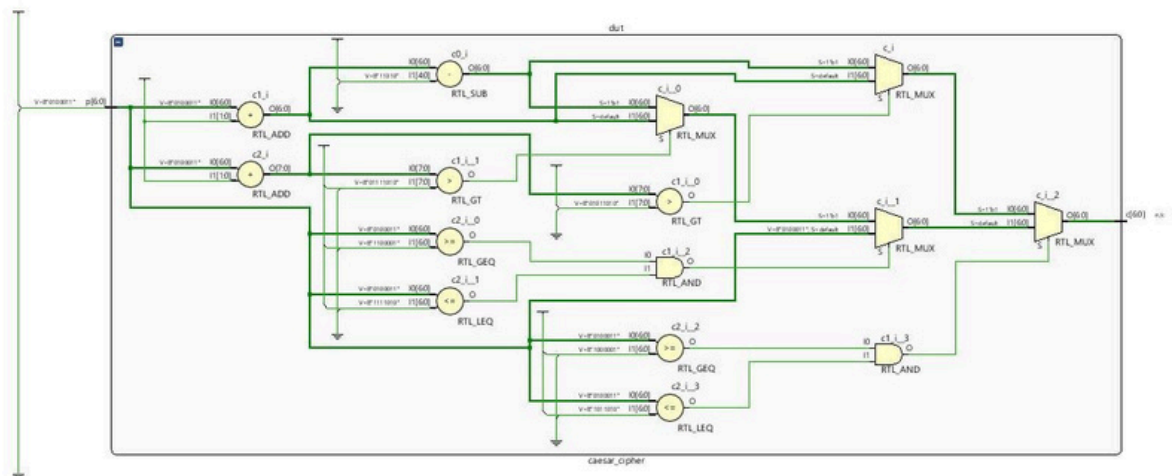
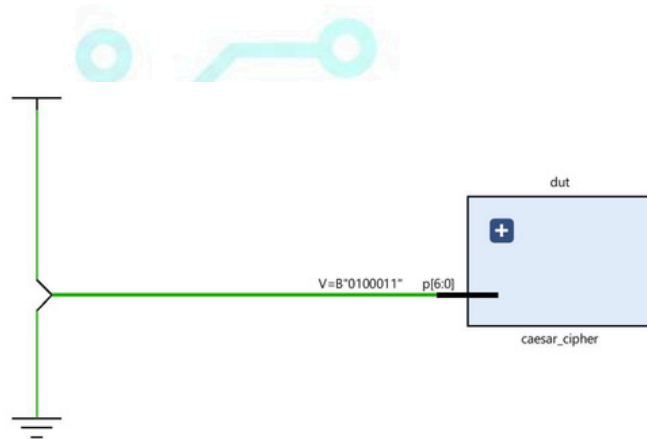


Tech & Teach

Task #6.3

3.3

```
module caesar_cipher(  
    input logic [6:0] p,  
    output logic [6:0] c  
);  
always_comb begin  
    if (p >= "A" && p <= "Z") begin  
        if (p + 3 > "Z")  
            c = p + 3 - 26;  
        else  
            c = p + 3;  
        end  
    else if (p >= "a" && p <= "z") begin  
        if (p + 3 > "z")  
            c = p + 3 - 26;  
        else  
            c = p + 3;  
        end  
    else  
        c = p;  
    end  
end  
endmodule
```



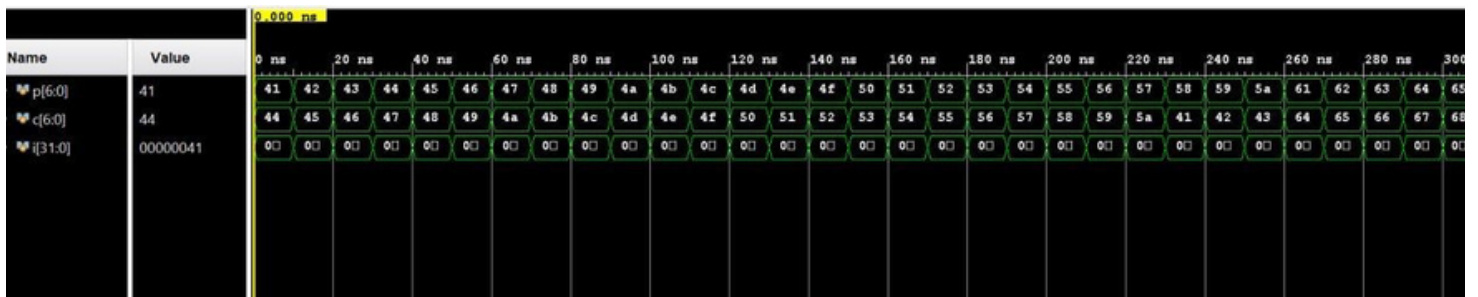
Task #6.3

3.4

```
`timescale 1ns/1ps
module caesar_cipher_tb;
logic [6:0] p;
logic [6:0] c;
caesar_cipher dut(
    .p(p),
    .c(c) );
integer i;
initial begin
    for(i="A"; i<="Z"; i=i+1) begin
        p = i;
        #10;
        $display("%c -> %c", p, c);
    end

    for(i="a"; i<="z"; i=i+1) begin
        p = i;
        #10;
        $display("%c -> %c", p, c);
    end

    p = "1"; #10; $display("%c -> %c", p, c);
    p = "@"; #10; $display("%c -> %c", p, c);
    p = "#"; #10; $display("%c -> %c", p, c);
    $finish;
end
endmodule
```



Work distribution

Section	Almas	Atheer	Mjd	Maha	Jood
Report	✓	✓	✓	✓	✓
Task6.1	✓	✓	✓	✓	✓
Task6.2	✓	✓	✓	✓	✓
Task6.3	✓	✓	✓	✓	✓

Tech & Teach